HPC-Driven Computer Vision: Accelerating Image Processing Stencils and Pointwise Operations via NVIDIA CUDA

University of Camerino, MSc in Computer Science, Parallel and Distributed Programming

1st Daniele Monaldi
School of Science and Technologies
University of Camerino
Camerino, Italy
daniele.monaldi@studenti.unicam.it

2nd Edoardo Papa
School of Science and Technologies
University of Camerino
Camerino, Italy
edoardo.papa@studenti.unicam.it

3rd Alessio Rubicini
School of Science and Technologies
University of Camerino
Camerino, Italy
alessio.rubicini@studenti.unicam.it

***Abstract*—This report examines the optimization and parallelization of three fundamental image processing algorithms: Sobel edge detection, Gaussian blur, and RGB-to-YUV color space conversion. We developed a progression of implementations, starting from single-threaded C++ baselines and evolving into CUDA solutions. The core objective was to bridge the gap between theoretical peak performance and actual throughput. For stencil-based operations, we implemented shared memory tiling and constant caching to mitigate the global memory bottleneck, reducing redundant DRAM fetches by nearly an order of magnitude. For pointwise transformations, we focused on maximizing memory bus saturation through a planar output layout and hiding PCIe communication overhead via asynchronous CUDA streams. Our results demonstrate that while algorithmic improvements like separability provide a solid foundation, the highest gains come from managing the memory hierarchy. We measured end-to-end performance using metrics such as throughput (MPixels/s) and effective bandwidth (GB/s). The final optimized kernels achieved significant improvements over optimized CPU versions, reaching the physical limits of the PCIe bus and driver launch latency. This analysis highlights the critical trade-off between compute density and data movement in modern high-performance computing environments.**

I. Introduction

A. Project Overview

Image processing is a core component of many modern applications, and the increasing demand for high-resolution data and real-time performance makes sequential implementations increasingly inadequate. GPU acceleration, and in particular NVIDIA CUDA, provides an effective platform to address these challenges by exploiting massive data parallelism and high memory bandwidth.

The objective of this project is to develop a high-performance image processing library using CUDA, focusing on both pointwise and stencil-based operations commonly found in computer vision pipelines. Rather than relying on naive GPU implementations, the project emphasizes the use of memory-aware optimization techniques to fully leverage the GPU architecture. Selected filters, including color space conversion, Gaussian blur, and Sobel edge detection, are used as case studies to analyze different computational complexities and memory access patterns. The project also explores the impact of algorithmic optimizations, such as the use of separable Gaussian blur and domain decomposition strategies, and evaluates performance through systematic benchmarking and profiling.

B. Motivation

Image processing algorithms are particularly well-suited for GPU acceleration due to the inherently data-parallel nature of image data. Most operations apply the same computation independently to each pixel or to small local neighborhoods, making them a natural fit for the Single Instruction Multiple Threads (SIMT) execution model used by modern GPUs. In pointwise operations, such as color space conversion, each output pixel depends only on the corresponding input pixel. This results in embarrassingly parallel workloads with minimal data dependencies. Similarly, stencil-based operations like Gaussian blur and Sobel edge detection apply the same convolution pattern across the image, allowing thousands of GPU threads to process different pixels concurrently.

Compared to CPUs, GPUs provide a much higher degree of parallelism and memory bandwidth, which can significantly reduce execution time when memory access patterns are carefully optimized. However, fully exploiting this potential requires an explicit awareness of the GPU memory hierarchy and execution model. This project is motivated by the need to understand how algorithm structure, data layout, and memory access patterns interact with the SIMT model to achieve high performance in practice.

II. Related Work

The optimization of image processing algorithms on GPU architectures is a widely researched field, particularly concerning the balance between computational throughput and memory bandwidth constraints. This chapter reviews existing

literature that informs our implementation of stencil operations, tiling strategies, and format conversions.

Stencil computations, which serve as the foundation for filters such as Sobel and Gaussian blur, are widely recognized as memory-bound. Sai et al. [1] show that high-order stencils introduce distinct optimization challenges compared to simpler kernels, largely due to complex data reuse patterns and the resulting load on the memory hierarchy. To alleviate the global memory bottleneck, adaptive tiling strategies have become standard. Van Werkhoven et al. [3] note that static tiling configurations often perform poorly because they do not consider hardware-specific constraints such as shared memory limits. Their work demonstrates that adjusting tile dimensions to match the occupancy limits of a GPU, such as the NVIDIA Tesla T4 used in our study, is essential for fully utilizing the memory bus.

A major concern in modern parallel programming is the so-called "Communication Wall," where the time spent transferring data over the PCIe bus exceeds the time spent on actual computation. In their study of high-performance edge detection, Yadav and Gupta [4] show that in multi-stage pipelines, the overhead of Host-to-Device transfers is the main limiting factor. Their experiments on the Tesla T4 architecture confirm the importance of using page-locked memory and concurrent CUDA streams. By overlapping kernel execution with data transfers, much of the latency can be hidden, preventing the Streaming Multiprocessors from sitting idle. This awareness of hardware behavior distinguishes a naive parallel implementation from a production-ready solution.

Another key topic in current literature is the trade-off between mathematical complexity and memory traffic. Mathematically, a separable Gaussian convolution is advantageous, reducing the operation count from $O(K^2)$ to $O(2K)$. However, Lu et al. [5] offer a more detailed view, showing that on high-bandwidth GPUs, the intermediate memory passes required by separable kernels can create more overhead than the arithmetic savings provide. Their work forms the theoretical basis for the "Gaussian Paradox" observed in our project, where a tiled 2D kernel can outperform a separable implementation when the latter is constrained by global memory transactions.

Finally, it's important to note that, as image processing increasingly targets distributed and real-time environments, the demand for robust abstractions and efficient data layouts becomes critical. Bhosale and Kale [2] investigate distributed stencil abstractions that automate boundary management, enabling parallel applications to scale across multiple nodes without manual halo handling. For localized operations such as color space conversion, the emphasis shifts to memory coalescing. Aguerre et al. [6] show that using planar memory layouts for YUV formats is essential for efficient real-time video processing. Their results indicate that memory structure directly determines the achievable saturation of the memory controller, a principle we applied in our optimized RGB2YUV implementation to reach near-peak bandwidth.

The findings presented in this literature review establish a clear framework for interpreting our experimental results, particularly concerning the dominance of memory-bound constraints over raw arithmetic throughput. By aligning our implementation strategies with documented research on tiling, asynchronous concurrency, and data layout efficiency, we demonstrate that maximizing performance on the Tesla T4 architecture is a matter of strategic resource management rather than simple parallelization. Ultimately, this context validates the performance trade-offs observed in our benchmarks, reinforcing the necessity of an architecture-aware approach to high-performance computer vision.

III. Background and Methodology

A. CUDA Architecture

The experimental evaluation in this project is performed on an NVIDIA Tesla T4 GPU, commonly available in cloud environments such as Google Colab. The T4 is based on the NVIDIA Turing architecture and is designed to provide a balance between computational throughput and energy efficiency for general-purpose GPU workloads. At the core of the CUDA programming model is the Streaming Multiprocessor (SM), which serves as the main execution unit of the GPU. Each SM is capable of executing thousands of lightweight threads concurrently by organizing them into groups called warps, each consisting of 32 threads executing the same instruction within the SIMT paradigm [7]. A distinctive feature of the Turing architecture [8] is its unified L1 Cache/Shared Memory unit, which can be dynamically partitioned to prioritize either hardware-managed caching or developer-controlled local storage. Efficient utilization of these resources is essential for achieving high performance, as they directly affect occupancy, memory latency, and overall throughput.

B. Memory Hierarchy

The performance of CUDA applications is strongly influenced by the GPU memory hierarchy, which provides different types of memory with varying latency, bandwidth, and scope.

- **Global memory** is the largest memory space available on the GPU and is accessible by all threads. However, it also has the highest latency (often exceeding 400-600 clock cycles) and limited cache effectiveness when access patterns are irregular. Naive implementations that rely heavily on global memory accesses often become memory-bound and fail to fully use the computational capabilities of the GPU.
- **Shared memory** is an on-chip memory region shared among threads within the same block. It offers significantly lower latency and higher bandwidth compared to global memory, making it well-suited for stencil-based operations such as convolutions. By cooperatively loading image tiles into shared memory, threads can reuse data efficiently and drastically reduce the number of global memory accesses.
- **Constant memory** is a small, read-only memory space cached on-chip and optimized for broadcast access. It is particularly effective when all threads read the same

values, as is the case for convolution kernels and filter coefficients. Using constant memory for such parameters reduces memory traffic and improves cache utilization.

In addition to on-device memory, data transfers between host and device can represent a significant performance overhead. To mitigate this, pinned (page-locked) host memory must be allocated to enable Direct Memory Access (DMA) transfers, allowing data to be copied directly between host and device without intermediate buffering. When combined with asynchronous transfers and CUDA streams, pinned memory enables the overlap of data movement and computation, further improving overall application throughput.

C. Evaluation Metrics

To evaluate the effectiveness of the proposed implementations, performance is measured using a set of quantitative metrics that capture both computational efficiency and memory behavior. These metrics are applied consistently across CPU and GPU versions to enable fair comparisons.

The primary metric is the average execution time, computed over multiple iterations to reduce the impact of noise and transient effects. For GPU versions, we explicitly distinguish between the pure 'Kernel Time' and the 'Total Pipeline Time,' the latter including the overhead of memory migrations. From execution time, throughput is computed as the number of processed pixels per second (MPixels/s). This metric is particularly useful for image processing workloads, as it provides a resolution-independent measure of performance and highlights scalability with respect to image size.

To assess memory efficiency, the effective memory bandwidth is estimated using the formula:

$$\text{Bandwidth (GB/s)} = \frac{\text{Bytes read} + \text{Bytes written}}{\text{Execution Time} \cdot 10^9}$$

Although this value does not represent peak hardware bandwidth, it provides insight into whether an algorithm is memory-bound and how efficiently it uses the memory subsystem.

Finally, speedup is used to compare optimized implementations against baseline versions, such as standard CPU or naive algorithms. Speedup values help quantify the impact of algorithmic optimizations, including separable convolutions and improved memory access patterns.

For CPU-based benchmarks, data transfer overheads between host and device are naturally absent. These measurements serve as a reference point for evaluating the benefits and trade-offs of GPU acceleration, which are analyzed in later sections.

IV. Software Architecture

A. Framework Design

The project is structured around a modular C++ framework designed to support a clear separation between algorithmic logic and low-level hardware management. We chose this approach to enable seamless comparisons between CPU and GPU implementations without duplicating boilerplate code. The system uses the single-header libraries `stb_image.h` and `stb_image_write.h` [13] for I/O operations. These tools were selected for their minimal footprint, allowing us to focus on computational kernels rather than complex image decoding pipelines. The software follows a CLI-driven model where parameters such as kernel size (K), blur intensity (σ), and the number of iterations can be passed at runtime. This flexibility was essential during the benchmarking phase, as it allowed us to move through different image resolutions and filter configurations to identify performance turning points.

B. RAII and Memory Management

Managing memory in CUDA often leads to fragmented code due to the manual nature of `cudaMalloc` and `cudaFree`. To address this, we implemented the Resource Acquisition Is Initialization (RAII) pattern via a custom `CudaMemoryManager` template class. This manager handles both device-side allocations and host-side pinned memory. By wrapping raw pointers in this class, we ensured that resources are automatically deallocated when the object goes out of scope, eliminating memory leaks during the iterative testing of large images. We chose to prioritize pinned memory (`cudaHostAlloc`) for all host-side buffers. While pinned memory consumes more system resources and can lead to performance degradation if over-allocated, it is a requirement for high-speed DMA transfers. In our project, the increased complexity of managing page-locked memory was a necessary trade-off to enable the asynchronous execution models used in the RGB-to-YUV module. During development, we noticed that using standard `std::vector` for large transfers introduced a significant hidden cost due to the driver internally copying data to a staging buffer before the actual PCIe transfer. Switching to the `CudaMemoryManager` with pinned support removed this intermediate step, resulting in more predictable H2D and D2H latencies.

C. Error Handling Strategy

Robustness in parallel programming is difficult to achieve because errors in asynchronous kernels often do not manifest until much later in the execution flow. Our strategy relies on the `gpuErrchk` macro, which wraps every CUDA API call and kernel launch. This macro performs an immediate check on the return status of the function and provides a detailed error string, the filename, and the line number upon failure.

For kernel launches, we followed each execution with `cudaPeekAtLastError` and a synchronization point during the debugging phase. We recognized that while constant synchronization allows for precise error localization, it introduces a significant performance penalty by stalling the pipeline. Consequently, for the final benchmarking builds, we limited explicit synchronization to the timing events, relying on the asynchronous error reporting for most of the processing. This balance ensured that the measured throughput reflects the real-world performance of the kernels rather than the

overhead of the monitoring framework. A minor issue we encountered was the driver-side timeout for very long-running kernels (TDR). We mitigated this by ensuring our work-groups were sized correctly for the Tesla T4's scheduler.

V. Image Processing Algorithms

A. Sobel Edge Detection

The Sobel Edge Detection algorithm is a widely used technique [9] in image processing for highlighting intensity variations and detecting edges. It is based on the computation of local image gradients and belongs to the class of stencil-based operations, where each output pixel depends on a small neighborhood of input pixels.

The sobel operator approximates the horizontal and vertical intensity gradients by convolving the input image with two fixed 3×3 kernels, typically denoted as G_x and G_y. The kernels emphasize intensity changes along the horizontal and vertical directions. For each pixel, the gradient magnitude is then computed by combining the two components, providing a measure of edge strength.

From a computational perspective, Sobel filtering exhibits regular memory access patterns but requires multiple reads per output pixel due to its stencil nature. Each pixel computation involves accessing neighboring pixels, which introduces data reuse opportunities but also makes the algorithm sensitive to memory latency if implemented naively. For GPU architectures, Sobel edge detection represents an ideal candidate to showcase the benefits of shared memory. By loading a tile of the image, including halo regions, into shared memory, multiple threads within a block can reuse the same data, significantly reducing redundant global memory accesses. This optimization is especially important for achieving high performance, as Sobel filtering is typically memory-bound rather than compute-bound.

B. Gaussian Blur

Gaussian blur is a fundamental image smoothing technique widely used in computer vision for noise reduction and scale-space analysis. It operates by convolving the input image with a Gaussian kernel, producing a weighted average of neighboring pixels that attenuates high-frequency components while preserving overall image structure.

In its standard formulation, Gaussian blur is implemented as a 2D convolution. For each output pixel, a weighted sum is computed over a $K \times K$ neighborhood, resulting in a computational complexity of $O(K^2)$ operations per pixels. While conceptually simple, this approach becomes increasingly expensive as the kernel size grows, both in terms of arithmetic operations and memory accesses.

A key property of the Gaussian kernel is its separability: a 2D Gaussian can be expressed as the product of two 1D Gaussians, one along the horizontal direction and one along the vertical direction. This allows the original 2D convolution to be decomposed into two successive 1D convolutions, first applied horizontally and then vertically. Exploiting this property reduces the computational complexity from $O(K^2)$ to $O(K)$ per pixel.

From a performance perspective, this reduction has significant implications. The separable formulation drastically lowers the number of arithmetic operations and memory accesses required for each output pixel. Moreover, the two-pass structure improves memory locality and reduces pressure on the memory subsystem, making the algorithm more efficient on both CPUs and GPUs.

In parallel architectures such as CUDA-enabled GPUs, Gaussian blur represents a representative stencil-based workload where performance is often limited by memory bandwidth rather than raw compute capability. For this reason, Gaussian blur, particularly in its separable form, is an ideal case study to analyze the interaction between algorithmic complexity, memory access patterns, and architectural optimizations.

C. Color Space Conversion (RGB-to-YUV)

Color space conversion from RGB to YUV is a classic pointwise image processing operation, where each output pixel depends solely on the corresponding input pixel. This characteristic makes the algorithm highly parallelizable and particularly well-suited for GPU execution under the SIMT model.

The conversion is defined by a linear transformation involving a fixed set of coefficients. While a straightforward implementation relies on floating-point arithmetic, this approach may introduce unnecessary overhead for a kernel that is primarily limited by memory throughput. To improve efficiency, the conversion can be implemented using fixed-point arithmetic, where coefficients are scaled to integers and normalization is performed through bit-shift operations. This reduces the number of floating-point instructions and register usage per thread, improving instruction throughput without compromising perceptual accuracy.

Memory layout proved to be the dominant factor for this kernel. A standard interleaved layout (RGBRGB...) prevents memory coalescing because threads in a warp access non-contiguous addresses. We transformed the data into a planar layout (YYYY... UUUU... VVVV...). This organization ensures that consecutive threads write to consecutive memory addresses. This change was the primary driver in saturating the memory bus, allowing the implementation to move closer to the theoretical peak bandwidth of the Tesla T4.

We recognize that while this specific layout maximizes kernel throughput, the overall benefit depends on the requirements of subsequent stages in an image processing pipeline. In our current implementation, this planar data is not further processed. However, this architectural choice provides a foundation for future vision tasks (such as neural network inference) where planar input is often a standard requirement.

VI. Parallel Optimization Strategies

A. Shared Memory Tiling

The main performance limitation of the naive stencil implementations was the high ratio between global memory

accesses and arithmetic operations. To address this issue, we implemented a tiling strategy that leverages the low-latency shared memory available on the Tesla T4. The image is partitioned into thread blocks, and threads within each block cooperatively load a data tile into a local `__shared__` array.

A critical aspect of this approach is the handling of the halo region. Since each output pixel in a convolution requires access to neighboring values, each block must load pixels beyond its nominal boundaries. We adopted a cooperative loading scheme in which threads are also responsible for loading the required halo pixels, applying coordinate clamping to manage boundary conditions. This design increases kernel complexity and introduces branch divergence during the loading phase. Nevertheless, it substantially reduces DRAM traffic. Each pixel is loaded from global memory once and reused by the stencil computations within the same block, reducing memory traffic and improving overall balance between memory access and computation. A `__syncthreads()` barrier is used to guarantee that all data are available in shared memory before the convolution phase begins, with the associated synchronization overhead accepted as necessary for correctness.

B. Constant Cache Exploitation

For both Sobel and Gaussian filters, the convolution masks and Gaussian weights are constant throughout kernel execution. Keeping these coefficients in global memory would result in repeated high-latency accesses by all threads. For this reason, the masks were moved to the `__constant__` memory space.

This choice takes advantage of the constant cache behavior on the Turing architecture. When all threads within a warp access the same constant memory location, the value is broadcast efficiently by the hardware. Although constant memory is limited to 64 KB, the filters used in this project require under 1 KB, making them suitable candidates. This change also reduced pressure on the L1 cache and slightly lowered register usage, which in turn improved warp residency on each SM.

C. Hiding Communication Latency

In the RGB-to-YUV conversion module, we observed that for simple pointwise kernels the PCIe transfer time can dominate overall execution time. To mitigate this communication overhead, we implemented a multi-stream pipeline based on asynchronous memory transfers. Synchronous `cudaMemcpy` calls were replaced with `cudaMemcpyAsync`, and the input image was divided into independent chunks.

Four CUDA streams were used to overlap Host-to-Device transfers, kernel execution, and Device-to-Host transfers on different chunks of the image. Pinned host memory was required to enable asynchronous DMA transfers. The benefit of this overlap depends on the chosen chunk size. When chunks are too small, the overhead introduced by stream scheduling becomes noticeable and limits performance. By selecting an appropriate chunk size, we reduced the overall execution time compared to a fully synchronous pipeline.

D. Domain Decomposition

To allow execution beyond a single GPU, the kernels were written using a simple domain decomposition scheme. Instead of assuming access to the full image, each kernel receives a `y_offset` and a `y_slice_height` parameter. On the host side, this makes it possible to split a high-resolution image (such as 4K or 8K) into horizontal slices that can be processed independently.

This design requires careful handling of global coordinates, especially for boundary conditions, so that clamping logic refers to the actual image limits rather than the local slice. All experiments in this work were performed on a single Tesla T4. However, the same structure can be extended to multi-GPU or MPI-based executions if needed. The only overhead introduced by this approach is a small increase in register usage to store the offset parameters.

VII. Results and Performance Analysis

A. Test Environment and Methodology

All benchmarks were run on a Google Colab instance equipped with an NVIDIA Tesla T4 GPU and an Intel Xeon CPU. To obtain reliable measurements, we first included a warm up run for each implementation, in order to avoid counting the one time overhead from CUDA context initialization and initial memory allocations. Average execution times were calculated over N iterations, specifically $N = 200$ for CPU and $N = 100$ for GPU kernels, in order to mitigate the impact of transient system noise. We isolated kernel performance using CUDA events for algorithmic analysis while measuring the full pipeline time to assess PCIe bus influence.

B. Pointwise and Stencil Scaling

The relationship between image resolution and execution time for pointwise and stencil operations remains strictly linear across our tests.

As shown in the plots for RGB-to-YUV scaling (see figure 1) and Sobel scaling (see figure 2) the execution time grows proportionally with the total number of pixels. This confirms that our GPU kernels effectively saturate the device's processing units as the workload increases.

Data for RGB-to-YUV conversion indicates that we reached a bandwidth limit early in the scaling process. For the 2048×2048 resolution, the GPU baseline kernel ran in 0.1181 ms, while the optimized planar version achieved 0.1131 ms. The proximity of these values proves that the conversion is already bandwidth-limited and close to the hardware throughput ceiling. While the planar layout simplifies independent channel processing in later pipeline stages, it brings almost no performance gain for the conversion step itself.

The plot in figure 7 shows the divergence between CPU and GPU performance of Sobel implementation. The sequential CPU baseline scaled from 40.79 ms to 645.48 ms as resolution increased. Meanwhile, GPU implementations maintained sub-millisecond execution times. We encountered a performance regression in the optimized shared-memory Sobel kernel. At

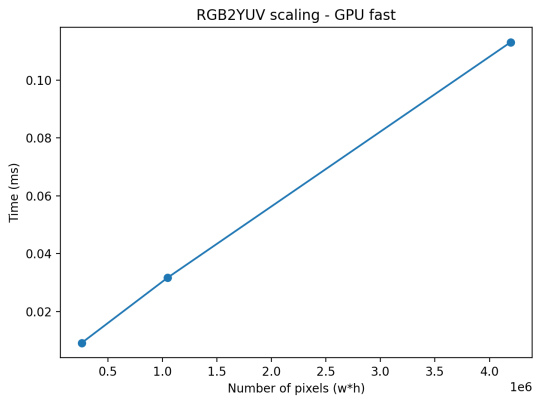

Fig. 1. RGB-to-YUV scaling

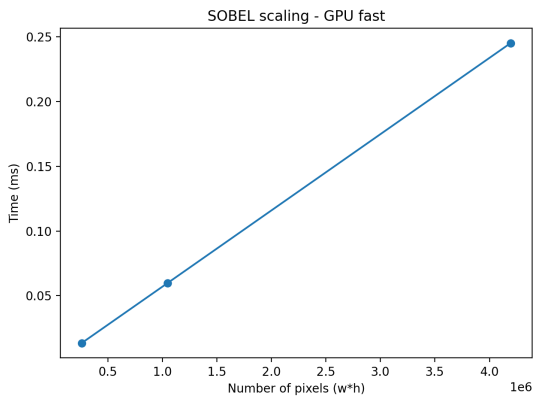

Fig. 2. Sobel scaling (GPU Fast)

2048×2048, the optimized version required 0.2452 ms, whereas the baseline GPU kernel finished in 0.1763 ms.

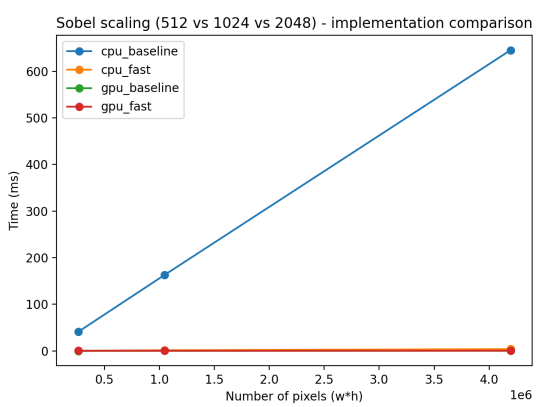

Fig. 3. Sobel scaling (all resolutions)

This result comes from a clear trade off between handling caching manually and relying on the GPU's built in cache. The 3×3 Sobel stencil does not involve much computation compared to the amount of memory it accesses. In the baseline version, the GPU's hardware cache already reuses data efficiently. When we switched to manual shared memory tiling, we added synchronization overhead and more complicated indexing to load the halo regions. For such a small 3×3 window, that extra management cost was higher than the savings from reducing global memory accesses, which explains the slowdown at high resolutions.

C. Gaussian Complexity Paradox

The Gaussian blur implementation provides the most significant deviation between theoretical algorithmic complexity and actual hardware performance. We evaluated the standard 2D convolution against the separable 1D passes across multiple kernel sizes K. The results highlight how memory behavior can dominate performance and reduce the practical benefits suggested by the lower theoretical complexity.

1) CPU and GPU Baseline Behavior: On the CPU, the separable implementation remains the only viable strategy as K increases. In the plot in figure 4, the 2D convolution follows a quadratic $O(K^2)$ growth, with execution time reaching 985.1239 ms for $K = 11$. Conversely, the separable version scales linearly, maintaining a much lower profile at 168.4606 ms for the same kernel size. The gap comes from performing far fewer arithmetic operations and making better use of the cache. In the 2D baseline, each pixel requires K^2 memory accesses, which quickly exceeds the capacity of the L1 and L2 caches as the kernel becomes larger.

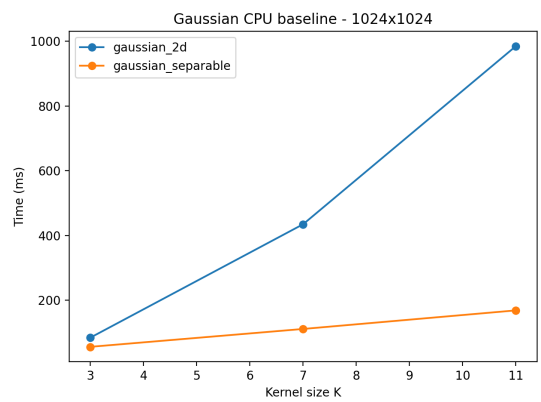

Fig. 4. Gaussian CPU Baseline (1024x1024)

The same behavior is initially visible in the GPU baseline (see figure 5). Without specific memory optimizations, the separable approach performs better because global memory bandwidth becomes the limiting factor. In the 2D GPU version, each output pixel still requires K^2 reads from global memory, generating a large amount of DRAM traffic that ultimately determines the execution time.

2) Shared Memory Inversion: The results from the optimized GPU implementation (see figure 6) contradict standard complexity theory. In this configuration, the optimized 2D kernel consistently performs better than the separable version. This turnaround happened because the system shifted from being compute-bound to being memory-bound.

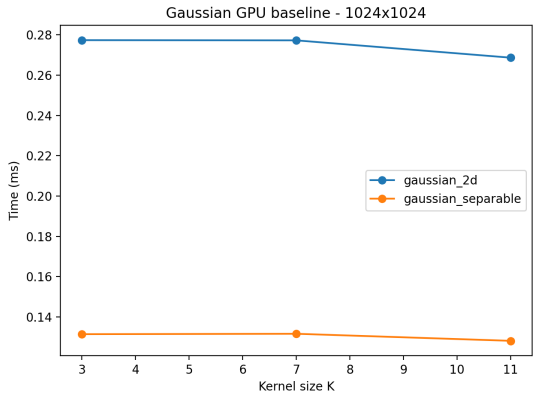

Fig. 5. Gaussian GPU Baseline (1024x1024)

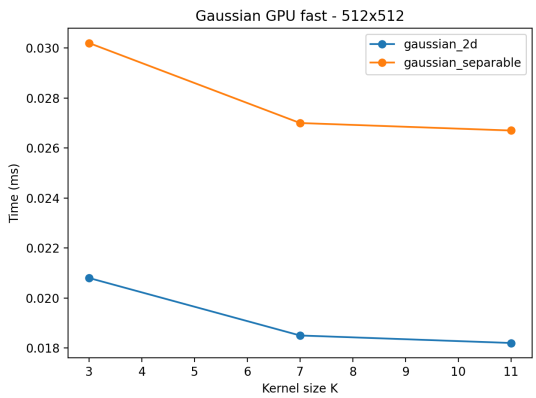

Fig. 6. Gaussian GPU Fast (512x512)

The optimized 2D kernel performs a single tiled convolution using shared memory, which limits global memory access to one read and one write. In contrast, the separable implementation requires two separate kernel launches and an intermediate buffer in global memory. We used the `float` format for this buffer to maintain precision, which increased global memory traffic by four times compared to the `uint8` format.

The main limitation moved from computational intensity to memory bandwidth. While the separable version requires fewer multiplications, $O(2K)$ versus $O(K^2)$, the cost of writing and then reading a large intermediate float buffer from global memory is greater than the cost of the additional arithmetic operations in the 2D tiled pass. During development, we observed that for kernel sizes up to 11×11, the overhead of the second kernel launch and the extra DRAM traffic makes the 2D version more efficient. This indicates that the separable version becomes meaningful only for much larger kernel sizes, where the $O(K^2)$ arithmetic cost eventually becomes larger than the memory overhead.

D. Throughput and Bandwidth Analysis

The test results clearly indicate the relative efficiency of each option (see table I). We calculated the effective bandwidth by measuring the total amount of data transferred through global memory and dividing it by the kernel execution time. This metric is the main indicator of how effectively our implementations use the Tesla T4 memory controller.

The throughput results highlight a huge gap between the sequential CPU baseline and the optimized GPU kernels. RGB2YUV achieved a peak throughput of more than $33,000$ MPixels/s. This level of performance is possible because the kernel is highly parallel and uses a planar layout that allows perfect memory coalescing. The effective bandwidth of 198.47 GB/s corresponds to about 62% of the Tesla T4 theoretical peak, which is an excellent outcome for a pointwise operation.

Stencil operations show lower throughput due to increased complexity. Sobel filtering achieved $17,534$ MPixels/s, while Gaussian blur 2D reached $8,893$ MPixels/s. The difference is primarily due to the larger memory footprint of the 7×7 Gaussian kernel compared to the 3×3 Sobel stencil. We observed that the separable Gaussian version reported a higher effective bandwidth (72.42 GB/s) than the 2D version (17.79 GB/s). This is not an indication of higher efficiency. Instead, it reflects the massive volume of data moved by the intermediate buffer. The 2D version is objectively faster in terms of wall-clock time because it avoids this redundant traffic entirely.

E. PCIe Overhead and Latency Analysis

The most important finding from our study is the clear impact of the so-called "Communication Wall." In our most optimized kernels, such as the GPU-accelerated RGB2YUV, the computation itself takes less than 0.1 ms. However, transferring the 1024×1024 RGB image over the PCIe bus requires several milliseconds. As a result, the GPU spends most of its time waiting for data instead of actually processing it.

To address this imbalance, we used asynchronous CUDA streams. By splitting the image into smaller chunks and using pinned memory, we were able to overlap the Host-to-Device transfer of chunk $N+1$ with the processing of chunk N. This approach masked a large portion of the data transfer latency. However, this optimization has its limits. When the chunks become too small, the overhead from launching many kernels and handling stream synchronization begins to undermine the performance gains.

Our analysis shows that GPU acceleration is not a universal solution. For small images or simple filters, the overhead of memory transfers and kernel launches can exceed the execution time of an optimized CPU implementation. The real benefit of the GPU becomes clear at higher resolutions and with more computationally intensive workloads. Indeed, at 2048×2048, the ratio between computation and data transfer becomes favorable, allowing the high throughput of the SMs to justify the cost of PCIe transfers.

VIII. Experimental Analysis

A. Block Size and Occupancy Tuning (Sobel)

Performance analysis of the Sobel kernel on the NVIDIA Tesla T4 highlights a strong dependence on the CUDA block geometry. Although the `dim3 blockSize(16,16)` configuration is often adopted as the standard choice for 2D filters,

TABLE I
COMPARATIVE PERFORMANCE METRICS FOR 1024×1024 RESOLUTION.

Implementation	Avg. Time (ms)	Throughput (MPix/s)	Eff. Bandwidth (GB/s)
Sobel CPU Naive	163.1003	6.43	0.01
Sobel CPU Fast	1.2636	829.83	1.66
Sobel GPU Naive	0.0637	16461.16	32.92
Sobel GPU Opt.	0.0598	17534.72	35.07
Gauss 2D CPU Naive	434.7314	2.41	0.00
Gauss 2D CPU Fast	198.0247	5.30	0.01
Gauss 2D GPU Naive	0.2773	3781.38	7.56
Gauss 2D GPU Opt.	0.1179	8893.77	17.79
Gauss Sep. CPU Naive	111.0698	9.44	0.09
Gauss Sep. CPU Fast	76.0997	13.78	0.14
Gauss Sep. GPU Naive	0.1316	7967.90	79.68
Gauss Sep. GPU Opt.	0.1448	7241.55	72.42
RGB2YUV CPU Naive	86.6562	12.10	0.07
RGB2YUV CPU Fast	29.1893	35.92	0.22
RGB2YUV GPU Naive	0.0336	31207.62	187.25
RGB2YUV GPU Opt.	0.0317	33078.11	198.47

experimental results show that, in our case, the 8×8 configuration provides the highest throughput, equal to 22,586.5 MPix/s, outperforming both the 16×16 (19,682.3 MPix/s) and the 32×8 (19,906.5 MPix/s) configurations, while the 32×32 configuration is clearly the worst (14,398.6 MPix/s).

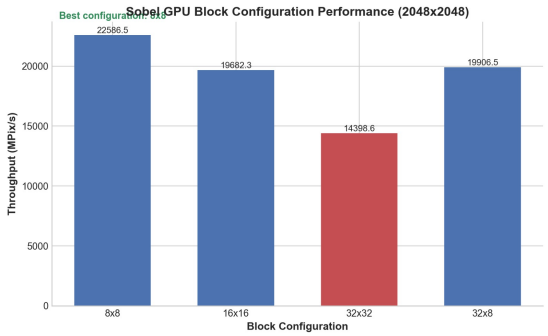

Fig. 7. Sobel GPU block configuration performance (2048x2048)

This behavior can be partially explained by looking at theoretical occupancy alone. On the Tesla T4 (Turing architecture, compute capability 7.5), each Streaming Multiprocessor (SM) supports at most:

- 1024 resident threads per SM;
- 32 resident warps per SM;
- 16 resident blocks per SM;
- 64K 32-bit registers per SM;
- 64KB shared memory per SM;

Indicating with:

- T_b = threads per block,
- $W_b = T_b/32$ = warp per block,
- R_t = registers per thread,
- S_b = shared memory per block,

the theoretical maximum number of active blocks per SM can be expressed as:

$$B_{active} = \min\left(\left\lfloor \frac{1024}{T_b} \right\rfloor, \left\lfloor \frac{32}{W_b} \right\rfloor, 16, \left\lfloor \frac{65536}{R_t \cdot T_b} \right\rfloor, \left\lfloor \frac{65536}{S_b} \right\rfloor\right)$$

From which the theoretical occupancy in terms of resident warps becomes:

$$\text{Occupancy} = \frac{B_{active} \cdot W_b}{32}$$

If we only consider thread/warp/block limits, and temporarily ignore register and shared memory constraints, the four tested configurations are all able to achieve 100% theoretical occupancy:

- 8×8: 64 threads/block, 2 warps/block, up to 16 blocks/SM $\Rightarrow$ 32 warps total;
- 16×16: 256 threads/block, 8 warps/block, up to 4 blocks/SM $\Rightarrow$ 32 warps total;
- 32×8: 256 threads/block, 8 warps/block, up to 4 blocks/SM $\Rightarrow$ 32 warps total;
- 32×32: 1024 threads/block, 32 warps/block, up to 1 block/SM $\Rightarrow$ 32 warps total.

This raises a key point: given the same theoretical occupancy, performance can vary significantly. Nominal occupancy alone, in fact, does not capture the kernel's actual efficiency because it does not describe the granularity of scheduling, the pressure on registers, the amount of shared memory consumed per block, nor the scheduler's ability to hide global memory latencies by alternating warps from different blocks.

1) Why 32×32 severely degrades performance: The 32×32 configuration contains 1024 threads per block, the entire resident thread budget available on a T4 SM. In practice, this means that each SM can host only one resident block. Although the 32 active warps formally saturate the theoretical occupancy, the scheduler loses flexibility: if a subset of warps

stalls due to global memory waits, there are no other resident blocks from which to extract alternative work. Furthermore, such a large block tends to simultaneously increase:

- register pressure,
- shared memory consumption per block,
- the risk of reducing actual occupancy if the compiler allocates many registers per thread.

For a kernel like Sobel, which requires neighbor access, local window loading, and gradient accumulation, this configuration is therefore too rigid and inefficient. The observed drop to 14,398.6 MPix/s confirms that a very large block is not necessarily advantageous, even when the theoretical occupancy would seem optimal.

2) Why 8×8 is the best configuration: The 8×8 configuration uses only 64 threads per block and allows, at least theoretically, up to 16 resident blocks per SM. This results in greater scheduling granularity and improved latency hiding: when some warps are waiting for data, the warp scheduler can more easily select ready warps belonging to other resident blocks. In other words, the 8×8 configuration not only maximizes parallelism, but also improves effective latency hiding.

An additional advantage is that smaller blocks tend to be less sensitive to the combined pressure of registers and shared memory, as they divide the resources of the individual SM more granularly. This makes the 8×8 configuration particularly suitable for stencil kernels or local filters, where the goal is not only to increase the number of threads but to maintain an execution pipeline flexible enough to absorb delays due to loads from global memory.

3) Interpreting the 32×8 Configuration: The 32×8 configuration deserves a separate discussion. While not the best, it slightly outperforms the 16×16 configuration. This is due to memory coalescing. With a block width of 32 threads, each warp can be naturally mapped along a row of the image, favoring contiguous accesses to global memory. In the presence of row-major layouts, this improves the likelihood that the memory requests of threads within a warp will be aggregated into more efficient DRAM transactions.

However, the data shows that this advantage is not sufficient to beat the 8×8 configuration. Therefore, in our Sobel kernel, coalescing along the x-axis is important, but it is not the only dominant factor. The better performance of 8×8 suggests that scheduling granularity and finer resource distribution across the SM are, in this case, more important than simply aligning the block width to the warp size.

Overall, the results show that on the Tesla T4, the block configuration cannot be chosen solely based on convention or theoretical occupancy alone. Although 16×16 represents a balanced and widely used choice, our study highlights that 8×8 is the true optimal operating point for the analyzed Sobel kernel, while 32×32 is clearly unfavorable. The comparison therefore demonstrates that:

1) theoretical occupancy is not sufficient to predict actual throughput;
2) configurations with smaller blocks can improve latency hiding;
3) warp-aligned configurations such as 32×8 favor coalescing but do not automatically guarantee the best result.

B. Optimizing Chunk Size and Stream Overlap for RGB2YUV

The multi-stream implementation of the RGB2YUV pipeline validates the efficiency of the overlap between PCIe transfer and GPU computation. Specifically, the end-to-end time per frame at 2048x2048 resolution decreases from 2.170ms in the single-stream version of the end-to-end implementation to 2.103ms in the version of the RGB2YUV pipeline using 1 stream, further decreasing to 1.679ms when 2 streams are used, 1.496ms when 4 streams are used, 1.412ms when 8 streams are used, and 1.400ms when 16 streams are used.

This progression indicates a strong scaling region from 1 to 4 streams and a saturation region for more than 8 streams. The relative improvement from the single stream configuration to the 4 stream configuration is:

$$\frac{2.170 - 1.496}{2.170} \cdot 100 \approx 31.1\%$$

whereas when scaling up from 4 to 16 streams, the additional gain is only:

$$\frac{1.496 - 1.400}{1.496} \cdot 100 \approx 6.4\%$$

The results show that the point of maximum practical benefit has already been almost entirely achieved with 4 streams.

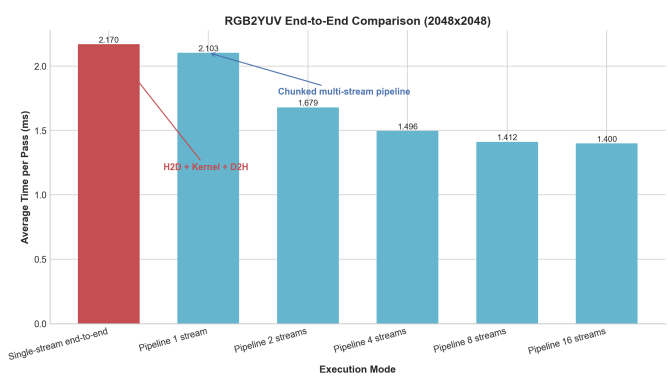

Fig. 8. RGB2YUV End-to-End Comparison (2048x2048 resolution).

1) Performance Analysis of Overlapped Execution: As illustrated in Figure 8, the transition from a single-stream implementation to a multi-stream pipeline yields immediate benefits. The average time per pass decreases from 2.170 ms to 2.103 ms with the introduction of a single asynchronous stream, with further significant gains as the number of streams increases.

Experimental results, further detailed in Figure 9, suggest that the actual **sweet spot** for this resolution lies between **4 and 8 streams**. Specifically, 4 streams provide the best trade-off in terms of performance gain versus implementation complexity. Beyond this point, the image is partitioned into excessively

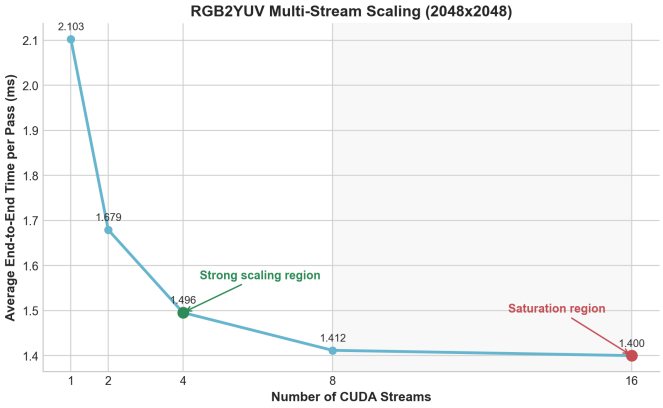

Fig. 9. Scaling analysis highlighting the strong scaling and saturation regions.

small chunks (e.g., approximately 128 rows per chunk for 16 streams), and the benefits of overlap are diminished by the overhead of orchestration.

If we are using **16 streams**, then the resolution is split into chunks of **approximately 128 rows per chunk**. In other words, every asynchronous transfer and kernel launch is performed on relatively small amounts of work. At that point, the total time is not dominated solely by the PCIe bus and kernel execution, but also by:

- kernel launch overhead;
- scheduling of asynchronous operations;
- management of stream queues by the CUDA driver;
- fragmentation of the work into tasks that are too small.

In qualitative terms, the total time can be modeled as:

$$T_{tot}(N_c) \approx \max\left(\frac{T_{H2D}}{N_c}, \frac{T_{kernel}}{N_c}, \frac{T_{D2H}}{N_c}\right) + N_c \cdot T_{sched}$$

where N_c represents the number of equivalent chunks or streams and T_{sched} the average overhead per asynchronous operation/launch. For small values of N_c, the first term dominates and it is advisable to increase parallelism; for large values of N_c, the second term increases to the point where further fragmentation becomes counterproductive.

2) Architectural implications for the Tesla T4: On the Tesla T4 with a **PCIe Gen3** interconnect, this behavior is entirely consistent with the existence of a threshold beyond which the latency of the CUDA SW stack and driver dominates the benefits of overlap. Hence, we may conclude that the optimal setting of the number of streams cannot be considered in isolation from the image sizes, as larger images allow for more chunks to be processed in parallel, whereas smaller images reach this limit much earlier.

For this reason, the result obtained for **2048x2048** pixels leads us to conclude that a setting of **4 streams** represents the optimal trade-off for our specific RGB2YUV processing task, whereas 8 or even 16 streams provide only marginal improvements and may not be worth the added complexity.

The results show that the multi-stream pipeline has the potential to provide significant end-to-end time reductions, but also that there exists a point beyond which no significant benefit can be achieved by introducing parallelism. Perhaps the most important contribution of this experiment, however, is the practical validation of the concept known as the "communication wall," where, after the PCIe transfer has been made almost negligible, the next wall to break is the launch overhead. In our hardware setup, for the resolution of 2048x2048 pixels, the optimal point appears to be in the range of 4-8 streams, with the preference being for 4 streams.

C. On the "Gaussian Paradox": Analysis and Interpretation

From the data, one can observe that the multi-stream pipeline is indeed capable of providing considerable reductions in time, but there is a point beyond which no tangible benefits are provided by further parallelism. However, the most important aspect of this experiment is that it provides a practical validation of the so-called communication wall, where, if the PCIe transfer time is almost completely hidden, then a new bottleneck is provided by the launch and scheduling time. Based on this, for this hardware configuration and this 2048x2048 resolution, the optimal point is in the 4-8 stream range, with 4 streams being the preferred choice.

From a theoretical point of view, a 2D forward Gaussian convolution requires a number of operations per pixel proportional to:

$$O(K^2)$$

whereas the separable version breaks the filter down into two 1D passes, reducing the cost to:

$$O(2K)$$

For this reason, in the absence of memory-aware optimizations, separability should almost always be beneficial for sufficiently large kernels. And this is indeed what we observe in the baseline version.

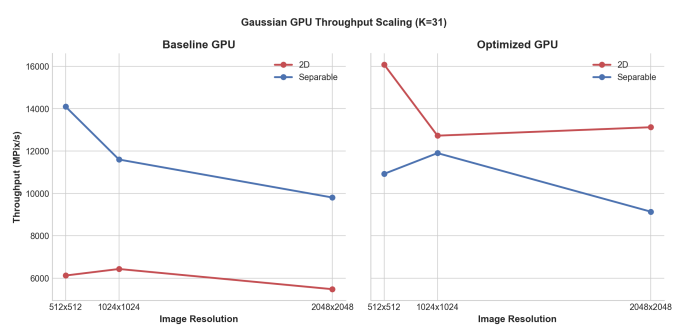

Fig. 10. Throughput comparison between Baseline and Optimized GPU implementations ($K = 31$). The optimized 2D version significantly outperforms the separable one.

1) Why the Optimized 2D Version Is More Efficient: The theoretical advantage of the separable pipeline, however, does

not take into account the cost of global memory traffic introduced by the intermediate buffer. As illustrated in Figure 10, while the baseline implementation (left) follows the theoretical advantage of the separable approach, the optimized version (right) completely reverses this trend. The separable pipeline actually requires:

1) horizontal pass;
2) writing the intermediate result to global memory;
3) reading the intermediate buffer;
4) vertical pass.

This means that the separable approach has an extra cost in terms of I/O operations in global memory, which can be avoided in the optimized version of the 2D algorithm by using shared memory, constant memory, or local tiling. In other words, the number of operations in the separable approach is reduced, but the number of off-chip operations is increased.

We can therefore summaries the total cost as follows:

$$T_{2D} \approx T_{load}^{tile} + T_{compute}^{K^2}$$

$$T_{sep} \approx T_{load}^{row} + T_{compute}^{K} + T_{store}^{interm} + T_{load}^{interm} + T_{compute}^{K}$$

The comparison between the two approaches therefore depends on the balance between:

- the computational cost of the K^2 operations in the 2D process;
- the cost of the additional data traffic required for the separable process.

2) Memory Bandwidth vs. Computational Complexity: The performance gap is explicitly quantified in Figure 11. In the optimized implementation, the ratio T_{sep}/T_{2D} remains consistently above the parity line (1.0), indicating that the 2D method is faster across all tested resolutions.

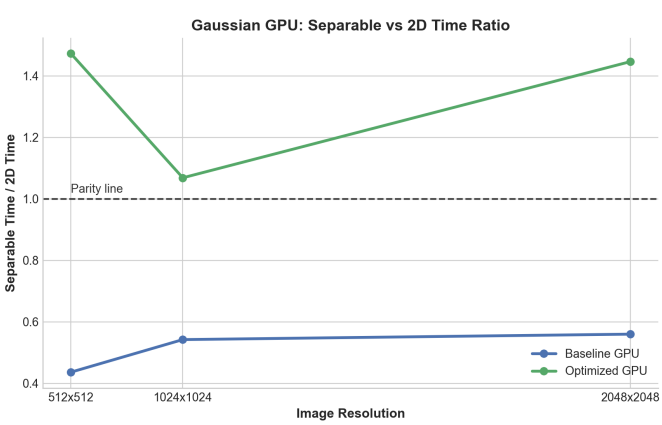

Fig. 11. Time ratio between Separable and 2D methods. Values above the parity line (1.0) indicate that the 2D method is faster.

The data shows that, for the Tesla T4 and K=31, the key factor in the optimised version is not the number of floating point operations, but the global memory bandwidth. In fact, a good implementation of the 2D tiled approach manages to reuse the loaded data and avoid accessing DRAM unnecessarily, whereas the separable approach has to compute an intermediate result. This explains the ratio:

$$\frac{T_{sep}}{T_{2D}} > 1$$

remains greater than one in the optimised version, with values reaching approximately **1.45** at the highest resolution.

In the case of a **2048×2048** image, the buffer with 'float' precision has a size of:

$$2048 \times 2048 \times 4; \text{byte} = 16{,}777{,}216; \text{byte} \approx 16; \text{MB}$$

This buffer must be written to global memory, read back from global memory, adding a total overhead which may easily dominate any benefit obtained from computation by moving from K^2 to $2K$ points, at least unless K is large.

3) Determining the Inversion Point (K^*): We define the inversion point K^* as the value of K for which:

$$T_{sep}(K^*) = T_{2D}(K^)$$

For values for which:

- $K < K^*$, the **optimised 2D** approach may still be faster, as the computational cost of the separable approach is still too high;
- $K > K^*$, the decrease in the complexity of the separable approach will be the dominant factor.

The results that have been found so far indicate that for the Tesla T4 and with an intermediate buffer in 'float', the value of K^* is **above 31** and in a higher part of the experimental domain.

4) Influence of FP16 Precision on Memory Bottlenecks: A natural extension of this analysis involves using FP16 (half-precision) for the intermediate buffer of the separable version. In this case, the buffer size is halved:

$$2048 \times 2048 \times 2; \text{byte} = 8{,}388{,}608; \text{byte} \approx 8; \text{MB}$$

This decrease in the amount of global traffic that is performed by writing to and reading from the buffer has a direct effect on the inflection point K^*. The idea is that if the cost of global memory is halved, the separable scheme becomes competitive sooner, simply because the arithmetic advantage is not so readily offset by the bandwidth cost.

In qualitative terms:

$$K^*_{FP16} < K^*_{FP32}$$

This statement is particularly pertinent with regards to Tesla T4, where support for mixed-precision computing makes this balance between computation and memory traffic potentially even more favorable for separable methods with very large kernels.

5) Synthesis: Resolving the Gaussian Paradox: The "Gaussian Paradox" is not a contradiction in the theory, but rather a result of the fact that the two approaches try to optimise different types of costs. The separable form minimises the number of arithmetic operations, while the optimised 2D form minimises the total amount of data traffic and takes better

advantage of the data locality. In the experimental regime, and especially for K = 31, the memory costs dominate the computational benefits of the separable form, and therefore the optimised 2D form remains faster. In order to experimentally determine the value of the inflection point K^*, it is necessary to extend the tests to larger kernels and to compare the float and half versions of the intermediate buffer.

IX. Conclusions

A. Final Remarks

This project examined the gap between theoretical algorithmic complexity and actual hardware throughput on the NVIDIA Turing architecture. By evolving our implementations from naive C++ baselines to optimized CUDA kernels, we identified that performance in modern computer vision workloads is rarely defined solely by arithmetic intensity, but rather by the efficiency of the memory hierarchy and the PCIe bus.

Our results on the RGB-to-YUV conversion demonstrate that pointwise operations are strictly bandwidth-bound. By adopting a planar output layout, we achieved an effective bandwidth of 198.47 GB/s, effectively saturating more than the half of the Tesla T4's memory controller. This confirms that for low-arithmetic intensity kernels, memory coalescing and layout optimization yield higher returns than instruction-level tuning.

Conversely, the Gaussian Blur analysis highlighted a "Complexity Paradox." While the separable formulation is theoretically superior ($O(K)$ vs $O(K^2)$), our benchmarks showed that the Optimized 2D Tiled implementation outperformed the separable version for standard kernel sizes (3×3 to 11×11). The overhead of the intermediate global memory buffer required by the separable filter proved more costly than the redundant arithmetic of the 2D kernel. This finding underscores that on GPUs, minimizing global memory traffic is often more critical than reducing the total number of floating-point operations.

Finally, the Sobel experiments revealed the trade-off of manual cache management. While shared memory tiling significantly reduced DRAM fetches, the synchronization overhead became a regression factor at very high resolutions compared to the hardware-managed L2 cache.

B. Potential Improvements

While our current implementations achieve significant speedups over CPU baselines, further architectural optimizations could push performance closer to the theoretical hardware limits:

- **Vectorized Loads (uchar4)**: currently, our kernels process image data using standard `unsigned char` pointers. A significant optimization would be to implement vectorized memory access using intrinsic data types like `uchar4` or `float4`. By loading a packed pixel (RGB or RGBA) in a single 32-bit transaction rather than three separate 8-bit reads, we could reduce the number of issued load instructions and improve instruction-level parallelism. This is particularly relevant for the RGB-to-YUV kernel, where increasing the memory transaction size could further improve bus utilization.
- **Advanced Asynchronous Pipelines (CUDA Graphs)**: we successfully employed CUDA Streams to hide PCIe latency by overlapping data transfers with kernel execution. However, as noted in our results, small chunk sizes introduce CPU-side launch overhead that can negate the benefits of concurrency. Future iterations could leverage CUDA Graphs. By capturing the sequence of memory transfers and kernel launches into a static graph, we could eliminate the runtime overhead of the driver's work submission. This would allow for much finer-grained concurrency, making the streaming approach viable even for lower resolutions or smaller chunk sizes.
- **Multi-GPU Scalability**: the domain decomposition logic currently implemented prepares the software for distributed execution. Extending this project to support Multi-GPU environments via NCCL (NVIDIA Collective Communications Library) would allow for real-time processing of 8K video streams, overcoming the memory capacity limits of a single device.

C. Code Availability

The source code is publicly available at: https://github.com/DarkShrill/GPUImageLab.git

REFERENCES

[1] R. Sai, M. Araya-Polo, J. Mellor-Crummey, J. Meng, e X. Meng, "Accelerating High-Order Stencils on GPUs," *arXiv preprint arXiv:2009.04619*, 2020. [Online]. Available: https://arxiv.org/abs/2009.04619

[2] A. Bhosale e L. Kale, "An Adaptive Distributed Stencil Abstraction for GPUs," in *2025 Supercomputing India (SCI)*, 2025. [Online]. Available: https://doi.org/10.1109/SCI68648.2025.11333865

[3] B. van Werkhoven, J. Maassen, e F. J. Seinstra, "Optimizing Convolution Operations in CUDA with Adaptive Tiling," *Future Generation Computer Systems*, vol. 30, pp. 14–26, 2014. [Online]. Available: https://www.semanticscholar.org/paper/Optimizing-Convolution-Operations-in-CUDA-with-Werkhoven-Maassen/9b1d3353fc3d6e1aee4ad6426d47187a321caa54

[4] R. Yadav e H. Gupta, "Optimizing CUDA Kernels for High-Performance Canny Edge Detection," *International Journal of Science, Engineering and Technology*, vol. 13, no. 4, 2025. [Online]. Available: https://www.researchgate.net/publication/395338017_Optimizing_CUDA_Kernels_for_High-Performance_Canny_Edge_Detection

[5] G. Lu, W. Zhang, e Z. Wang, "Optimizing Depthwise Separable Convolution Operations on GPUs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 70–87, 2021. [Online]. Available: https://eprints.whiterose.ac.uk/id/eprint/174797/1/main.pdf

[6] J. Aguerre, B. Sena, e D. Stolarz, "Using GPU-accelerated pixel format conversions for efficient real-time video streaming," *SMPTE Motion Imaging Journal*, vol. 133, no. 5, pp. 60–70, 2024. [Online]. Available: https://www.researchgate.net/publication/384074258_Using_GPU-accelerated_pixel_format_conversions_for_efficient_real-time_video_streaming

[7] NVIDIA Corporation, "CUDA C++ Programming Guide," *NVIDIA Documentation*, ver. 12.0, 2023. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[8] NVIDIA Corporation, "NVIDIA Turing GPU Architecture: Whitepaper," *NVIDIA Corporation*, 2018. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

[9] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[10] NVIDIA Corporation, *CUDA C++ Best Practices Guide*, v12.4, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[11] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.4, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[12] NVIDIA Corporation, *NVIDIA Turing GPU Architecture Tuning Guide*, 2019. [Online]. Available: https://docs.nvidia.com/cuda/turing-tuning-guide/index.html

[13] S. Barrett, "stb_image.h - v2.28 - public domain image loader," *GitHub Repository*, 2023. [Online]. Available: https://github.com/nothings/stb